

EPIC 06 — Database & Distributed Data Design

Jira Task 02: Database Indexing, Query Optimization & Performance

Sprint: Sprint 12 (21/9 - 4/10)

Priority: High **Effort:** 8 Hours

Objective: Learn how to identify slow database operations and improve query performance, using the order_db database with a realistic sample dataset.

Contents

- 1. Sample dataset
- 2. Query tested
- 3. Single-column index: before vs after
- 4. Composite index: user_id + created_at
- 5. Query optimization practices applied
- 6. Reasoning behind each change
- 7. Deliverables

1. Sample dataset

10,000 synthetic order records were inserted into `order_db.orders` using a single SQL script, with randomized `user_id`, `product_id`, `quantity`, `amount`, `status`, and `created_at` values spread across the past year. This gave a realistic volume of data large enough to observe genuine query-performance differences.

```
INSERT INTO orders (user_id, product_id, product_name, quantity, amount, status,
created_at)
SELECT ...
FROM (SELECT 1 FROM information_schema.columns LIMIT 100) t1,
     (SELECT 1 FROM information_schema.columns LIMIT 100) t2;
```

Result: 10,000 rows affected. Records: 10,000, Duplicates: 0, Warnings: 0.

2. Query tested

The query below is representative of a common lookup pattern: fetching all orders belonging to a given user.

```
SELECT * FROM orders WHERE user_id = 250;
```

3. Single-column index: before vs after

Before index (EXPLAIN result)

Metric	Value
type	ALL
possible_keys	NULL
key	NULL
Meaning	Full table scan - every row checked

Index created:

```
CREATE INDEX id_orders_user_id ON orders(user_id);
```

After index (EXPLAIN result)

Metric	Value
type	ref
possible_keys	id_orders_user_id
key	id_orders_user_id
Meaning	Index lookup - only matching rows accessed

Result: the query plan changed from a full table scan (type = ALL) to an indexed lookup (type = ref). On a table of this size the difference is already measurable, and it grows sharply as the table scales into the millions of rows.

4. Composite index: user_id + created_at

A composite index was added to support queries that filter by both user and a date range at once - a common real-world pattern (e.g. "this user's orders from last month").

```
CREATE INDEX idx_orders_user_created ON orders(user_id, created_at);
```

Query tested:

```
SELECT * FROM orders WHERE user_id = 250 AND created_at > '2026-06-01';
```

EXPLAIN result with composite index

Metric	Value
type	range
possible_keys	idx_orders_user_created
key	idx_orders_user_created
Meaning	Index used to locate the user, then range-scan on date

Why column order matters

A composite index is sorted by its first column, then its second column within each first-column group - the same way a phone book is sorted by last name, then first name. An index on (user_id, created_at) can efficiently answer "all orders for user 250" (uses user_id alone) or "user 250's orders after a date" (uses both columns in order). It cannot efficiently answer "all orders after a date" for an arbitrary user, because created_at is not the leading column. user_id was placed first because it is the more selective, more frequently filtered-alone field in this application.

5. Query optimization practices applied

Avoid SELECT *

Only the columns actually needed are selected, reducing data transfer and memory use.

```
SELECT id, amount, status FROM orders WHERE user_id = 250;
```

Pagination

Large result sets are fetched in pages rather than all at once.

```
SELECT id, amount, status FROM orders
WHERE user_id = 250
ORDER BY created_at DESC
LIMIT 20 OFFSET 0;
```

Filtering at database level

Filtering happens in SQL (WHERE), not by pulling all rows into application code.

Avoiding N+1 queries

Batch related lookups instead of issuing one query per row in a loop (e.g. calling UserServiceClient once per order).

Proper joins

Related data within the same database is combined with JOIN instead of separate round-trips.

```
SELECT o.id, o.amount, oi.product_name
FROM orders o
JOIN order_items oi ON oi.order_id = o.id
WHERE o.user_id = 250;
```

Query plans

EXPLAIN was used throughout this task to verify how each query actually executes.

6. Reasoning behind each change

- A single-column index on `user_id` was added because `user_id` is the most common filter across order queries (fetching one user's orders is the dominant access pattern).
- The composite index on (`user_id`, `created_at`) was added to support date-range filtering scoped to a single user, without needing a second index.
- No index was added on `created_at` alone, since it is rarely filtered without also filtering by user in this application's access patterns.
- Query optimization practices (avoiding `SELECT *`, pagination, proper joins) were reviewed against the existing codebase - order-service already avoids N+1 patterns by using a single `UserServiceClient` call per request rather than looping.
- `EXPLAIN` was used as the verification tool throughout, ensuring every claimed improvement is backed by an actual query plan change, not just an assumption.

7. Deliverables

Item	Status / location
Sample dataset	10,000 rows inserted into orders (Section 1)
Index definitions	<code>id_orders_user_id</code> , <code>idx_orders_user_created</code> (Sections 3-4)
Query optimization examples	<code>SELECT *</code> avoidance, pagination, joins, N+1 avoidance (Section 5)
Query-plan evidence	<code>EXPLAIN</code> output captured before and after each index (Sections 3-4)
Performance comparison report	This document
Updated database documentation	Indexes recorded against the <code>order_db</code> schema from Task 01